Style Guide (applies to both books)

Reader

A Manager, Data Specialist at a commercial pharma company. They sit between business stakeholders (Brand, Sales Ops, Market Access, Content Generation, Medical) and the data engineering team. They write data product specs, source-to-target mappings, data dictionaries, DQ rules and SLAs, manage the engineering backlog, and support data scientists / ML engineers. Tech stack is Snowflake. They know SQL, relational concepts, and ETL/ELT basics. They are NOT a software engineer, but they need to understand architectures deeply enough to (a) talk credibly with architects and engineers, (b) write specs that are implementable, and (c) judge whether a proposed design is fit for purpose.

Assume intelligence, not prior infrastructure knowledge. Explain every term the first time it appears.

Voice

- Plain, direct, concrete. Teach like a senior architect mentoring a smart colleague.
- Prefer "here is how it works, here is why, here is where it breaks" over abstract lists.
- Every major concept gets at least one concrete example from commercial pharma data (HCP/HCO, IQVIA APLD/NPA/NSP/Xponent, Symphony Health, Veeva CRM, claims, market access, field force / NBA, MLR content review, Medical). Use realistic but invented numbers.
- Where Snowflake has a relevant native feature, name it and explain what it does. If a feature detail is version-dependent or you are unsure, say "check current Snowflake docs" rather than invent.
- Do not hype AI. Be honest about where it fails and what it costs.

Chapter structure (use exactly this skeleton)

````
# Chapter N: Title

> One-paragraph framing: what problem this chapter solves and why the reader should
care.

## What you will learn
- 4–6 bullets

## N.1 ... (body sections; as many as needed)
   Use ### sub-sections freely. Include Mermaid diagrams (```mermaid) for architectures
and flows.
   Include tables for comparisons. Include short SQL / YAML / pseudo-code where it
clarifies.

## Worked example
   A single end-to-end pharma-commercial example that exercises the chapter's ideas.

## For your role
   3–6 bullets: how this chapter changes what you do day to day (specs you write,
questions you
   ask engineers, red flags to catch in design reviews, acceptance criteria to add).

## Design review checklist
   8–15 yes/no questions a reader can use when reviewing a design.

## Key takeaways
   5–8 bullets.

## Exercises
   3–5 exercises. At least one should be "write a spec / ticket / mapping for X".

## Further reading
   Real books, papers, docs. No invented URLs; give titles and authors, or well-known
doc sites.
````

Length

Target 4,000-6,500 words per chapter. Depth beats breadth. Do not pad.

Formatting rules

- Markdown. Mermaid for diagrams. ASCII tables are fine as an alternative to Mermaid.
- Cross-reference other chapters as "see Chapter 7 (Model Serving)" using the TOC in the book's README.
- Consistent terms: "data product", "semantic layer", "feature store", "RAG", "HCP", "HCO".
- Define acronyms on first use within each chapter (chapters may be read standalone).

- No em-dashes. Use commas, periods, or parentheses.

Inline examples (added after reader feedback)

The reader learns best from examples. Every new concept gets a short concrete example close to where it is introduced. Use this format so examples look the same across both books:

```
> **Example: <short title>**
> 2–8 lines of explanation with pharma-commercial specifics. A small snippet (JSON,
YAML, SQL, or
> pseudo-code, at most ~15 lines) may follow the blockquote in a fenced block.
> Full runnable version: Book 1 Appendix C, Example C3 (if one exists).
```

Inline snippets are illustrative and short. Full, tested code lives only in the code companions.

Code companions (the "working code" section)

Working code is kept separate so it can be skipped on a first read. - Book 1: runnable code in `book1-ai-system-design/code/`, walked through in `book1-ai-system-design/appendix-c-working-code.md`. Node.js (ESM JavaScript, no build step), tests with `node --test`, mock mode by default so nothing needs credentials. - Book 2: code in `book2-data-system-design-for-ai/code/`, walked through in `book2-data-system-design-for-ai/appendix-d-working-code.md`. Snowflake SQL, Terraform, and standard-library Python with local tests. Snowflake SQL and Terraform cannot be executed in the authoring environment, so each file states whether its syntax was checked against current docs.

Example IDs (use these exact IDs in cross-references)

Book 1, Appendix C: - C1 `code/c1-mcp-server`: MCP server exposing HCP data-product tools with entitlement filtering - C2 `code/c2-mcp-gateway`: MCP gateway (authn, per-role tool allow-lists, rate limits, PII redaction, audit log, kill switch, approved-server registry with pinned tool-description hashes) - C3 `code/c3-delegated-identity`: mock identity provider, OAuth token exchange to a short-lived, downscoped "on behalf of" agent token, verified by the gateway - C4 `code/c4-a2a-agents`: two agents talking over an A2A-style protocol (agent card + JSON-RPC) - C5 `code/c5-llm-gateway`: LLM gateway with model tiering, fallback, budgets, cost attribution, caching - C6 `code/c6-traced-agent-loop`: tool-using agent loop with step/token budgets and OpenTelemetry traces - C7 `code/c7-versioning-and-promotion`: tool/prompt registry with semantic versioning, breaking-change detector for tool schemas, eval gate, and a CI workflow promoting dev to test to prod

Book 2, Appendix D: - D1 `code/d1-agent-access-setup`: Snowflake service user, roles, grants, and policies for agents - D2 `code/d2-semantic-view-and-cortex-agent`: semantic view, Cortex Search service, Cortex Agent - D3 `code/d3-snowflake-mcp-server`: exposing governed tools through Snowflake's managed MCP server - D4 `code/d4-external-access`: network rule, secret, external

access integration (NPI Registry lookup) - D5 `code/d5-spcs-gateway`: hosting the MCP gateway in Snowpark Container Services - D6 `code/d6-terraform-environments`: Terraform for dev/test/prod Snowflake environments - D7 `code/d7-interface-versioning`: data-product and tool contract versioning checker (Python, tested)